

Devoir surveillé d'informatique

⚠ Consignes

- Les programmes demandés doivent être écrits en C. On suppose que les bibliothèques standard usuelles (`<stdio.h>`, `<stdlib.h>`, `<stdbool.h>`, `<assert.h>`, ...) sont déjà importées.
- On pourra toujours librement utiliser une fonction demandée à une question précédente, même si cette question n'a pas été traitée.
- Veillez à présenter vos idées et vos réponses partielles, même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

□ Exercice 1 : Questions de cours

On considère l'algorithme suivant :

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```

1  $r \leftarrow 0$ 
2 tant que  $m > 0$  faire
3   |    $m \leftarrow m - 1$ 
4   |    $r \leftarrow r + n$ 
5 fin
6 return  $r$ 
```

- Q1–** Donner les valeurs successives prises par les variables m , n et r si l'on exécute cet algorithme avec $n = 7$ et $m = 4$. On pourra recopier et compléter le tableau suivant :

	n	m	r
valeurs initiales	7	4	0
après un tour de boucle	...	...	...
après deux tours de boucle	...	...	...
après trois tours de boucle	...	...	...
après quatre tours de boucle	...	...	...

- Q2–** Donner une implémentation de cet algorithme en langage C sous la forme d'une fonction `multiplie` de signature `int multiplie(int n, int m)`. On précisera soigneusement la spécification de cette fonction en commentaire dans le code et on vérifiera les préconditions à l'aide d'instructions `assert`.
- Q3–** Donner la définition d'un variant de boucle, puis prouver que cet algorithme termine.
- Q4–** Donner la définition d'un invariant de boucle, puis prouver que cet algorithme est correct.

□ Exercice 2 : Retourner un tableau

Dans cet exercice, on s'intéresse à un algorithme permettant de « retourner » un tableau, c'est-à-dire réorganiser l'ordre de ses éléments de façon que le premier élément devienne le dernier, le deuxième devienne l'avant-dernier et ainsi de suite. Par exemple, le tableau $\{2, 7, 1, 9, 3\}$ deviendrait $\{3, 9, 1, 7, 2\}$. On notera bien que le retournement doit s'effectuer *en place*, c'est-à-dire qu'on ne veut pas créer de nouveau tableau mais simplement modifier le tableau existant. En notant t_0 le tableau initial et t_r le tableau « retourné », on a donc pour tout $k \in \llbracket 0; n-1 \rrbracket$, $t_r[k] = t_0[n-1-k]$. On propose pour cela l'algorithme suivant :

Algorithme : Retourner un tableau**Entrées** : Un tableau t de taille n **Sorties** : Aucune**Résultat** : Le tableau t est modifié (retourné)

```

1  $i \leftarrow 0$ 
2  $j \leftarrow n - 1$ 
3 tant que  $j - i > 0$  faire
4   | échanger les éléments d'indices  $i$  et  $j$  dans  $t$ 
5   |  $i \leftarrow i + 1$ 
6   |  $j \leftarrow j - 1$ 
7 fin

```

- Q5**– Montrer que cet algorithme termine.
- Q6**– Montrer que la propriété suivante, notée I , est un invariant de l'algorithme : $i + j = n - 1$.
- Q7**– Montrer que cet algorithme est correct. On pourra pour cela considérer l'invariant I' : « le tableau t contient les valeurs de t_r (le tableau retourné) pour tous les indices $k \in \llbracket 0; i - 1 \rrbracket \cup \llbracket j + 1; n - 1 \rrbracket$ » et utiliser l'invariant I de la question précédente.
- Q8**– On souhaite utiliser cet algorithme afin d'écrire en langage C une fonction `retourner_str` qui inverse une chaîne de caractères (par exemple, la chaîne "MP2I" devient "I2PM"). Rappeler la façon dont sont implémentées en C les chaînes de caractères et expliquer pour quelle raison il n'est pas utile de fournir la longueur de la chaîne en paramètre de la fonction `retourner_str`.
- Q9**– Écrire une fonction de signature `void echange(char s[], int i, int j)` qui échange dans s les caractères situés aux indices i et j .
- Q10**– Écrire une implémentation de la fonction `retourner_str` sans utiliser les fonctions de la bibliothèque `<string.h>`.
- Q11**– On rappelle qu'un palindrome est un mot qui se lit de la même façon de droite à gauche ou de gauche à droite, par exemple « radar » est un palindrome, mais « tata » n'en est pas un. Pour tester si un mot est un palindrome, un élève propose la solution suivante :

```

1 bool palindrome(char s[])
2 {
3     //renvoie true ssi s est un palindrome
4     char copie[] = s;
5     retourner_str(copie);
6     return s==copie;
7 }

```

Ce programme ne compile pas et produit une erreur à la ligne 4 : « *error : invalid initializer* ». Expliquer la cause de cette erreur (on ne demande pas de réécrire la fonction `palindrome`, mais d'expliquer de façon succincte l'erreur commise à la ligne 4 et la façon d'y remédier).

- Q12**– On suppose corrigée l'erreur commise à la ligne 4. Cette fonction est-elle alors correcte ? Justifier votre réponse.

□ **Exercice 3** : Autour de la recherche de minimums

On souhaite écrire une fonction de signature `int min(int tab[], int taille)` qui prend en entrée un tableau non vide d'entiers `tab` ainsi que sa taille `taille` et renvoie le minimum des éléments de ce tableau. On donne ci-dessous deux réponses d'élèves où les fonctions sont nommées respectivement `min_eleve1` et `min_eleve2`.

```

1  int min_eleve1(int tab[], int taille)
2  {
3      // Renvoie le minimum des éléments de tab (supposé non vide)
4      assert(taille > 0);
5      int cmin = 0;
6      for (int i = 0; i < taille; i++)
7      {
8          if (tab[i] < cmin)
9          {
10             cmin = tab[i];
11          }
12      }
13      return cmin;
14  }

```

```

1  int min_eleve2(int tab[], int taille)
2  {
3      // Renvoie le minimum des éléments de tab (supposé non vide)
4      assert(taille > 0);
5      int cmin = tab[0];
6      for (int i = 0; i < taille - 1; i++)
7      {
8          if (tab[i + 1] < tab[i])
9          {
10             cmin = tab[i + 1];
11          }
12      }
13      return cmin;
14  }

```

- Q13**– Donner un exemple de tableau à 3 éléments pour lequel `min_eleve1` renvoie un résultat correct.
- Q14**– Proposer un exemple de votre choix montrant que `min_eleve1` ne répond pas à sa spécification.
- Q15**– Donner un exemple de tableau à 3 éléments pour lequel `min_eleve2` renvoie un résultat correct.
- Q16**– Proposer un exemple de votre choix montrant que `min_eleve2` ne répond pas à sa spécification.
- Q17**– Donner une version de la fonction `min` conforme à sa spécification.
- Q18**– On veut maintenant obtenir, en plus du minimum, son nombre d'occurrences dans le tableau. Par exemple, pour le tableau {1, 0, 7, 2}, le minimum est 0 (il apparaît 1 fois) et pour {1, 7, -3, 2, -3, 4}, le minimum est -3 (il apparaît 2 fois). Pour cela, on propose d'écrire une fonction de signature : `int min_occ(int tab[], int taille, int *nb)`. Cette fonction renvoie le minimum *et* modifie la valeur pointée par `nb` afin qu'elle contienne le nombre d'apparitions du minimum. Donner une implémentation en langage C de la fonction `min_occ`.

On s'intéresse maintenant à la recherche des deux plus petites valeurs d'un tableau contenant au moins deux éléments (ces deux valeurs pouvant être égales si le minimum apparaît au moins deux fois).

- Q19**– Écrire un algorithme (en pseudo-langage) permettant de résoudre ce problème. On écrira une instruction `return` pour renvoyer les deux plus petites valeurs.
- Q20**– En C, on ne peut pas directement renvoyer deux entiers, aussi on propose de passer à la fonction deux pointeurs vers des entiers dont le contenu sera mis à jour afin de contenir après appel les deux plus petites valeurs. Écrire cette fonction.
- Q21**– Proposer un invariant de boucle qui permettrait de prouver la correction de cet algorithme. On ne demande pas de faire une preuve de correction, mais simplement d'énoncer un invariant préservé durant le déroulement de l'algorithme.